Visual Studio and Testing

CS 3500 - Lab 1

Last Updated: Fall 2024

Authors: Daniel Kopta, H. James de St. Germain

1 Overview

In this lab you'll practice utilizing the Visual Studio Integrated Development Environment (IDE). You will also be introduced to the idea of Test Driven Design (TTD) where you create tests for code (actually for interfaces) before you even write the code!

2 Lab Learning Outcomes

At the end of the lab, students should:

- Be able to create a new Project/Solution in Visual Studio
    - Know the difference between Projects and Solutions
- Know about dependencies and how to use a dll to add functionality to a project
- Know how to create and name a basic unit test
- Know some of the **Attributes** associated with C#'s MSTest Framework

3 Lab Worksheet

All labs will require that you complete a worksheet to show your understanding. The worksheet will be provided via a Canvas Quiz. Go to the main course page and open the lab1 worksheet now. You will be instructed when to enter your answers.

4 "It's Who You Know"

Before the lab starts and/or during the first 5 minutes, form groups of three people. **Choose people who you do not already know**. Introduce yourself to each of the others. Tell them where you are from, where you took CS 2420, and what your favorite piece of software is.

You are **not** allowed to sit quietly. Get up! Mingle.

5 Welcome to Visual Studio

You'll be spending a lot of time using Visual Studio this semester, and you must install it on your own computer[1].

Note: You are expected to already have Visual Studio installed on your laptop.

1. If you have not installed Visual Studio on your own laptop yet, you will need to use the CADE remote desktops during the lab today.

2. If you are having trouble installing Visual Studio, you can use the CADE remote virtual machines for now by following these instructions. Warning: if you are off campus, you must first log in to the Utah VPN, see these instructions.

   Important: Make sure you do your work in your home directory somewhere (the X:\ drive).

6 Visual Studio Solution

In this lab, we'll be looking at C# and Visual Studio's support for **unit testing**. Your first homework assignment is all about unit testing (and Test Driven Design) so we'll use the "Assignment One Start Code" as our starting point for this lab.

1. Download the Assignment One Starter Code from Canvas (under week 1).

2. Unzip this file and extract the contents to a new folder

   IMPORTANT: you need to actually right-click → extract the zip, don't just double-click on it.

   If you are on a CADE virtual machine, extract it into your home directory (X:\).

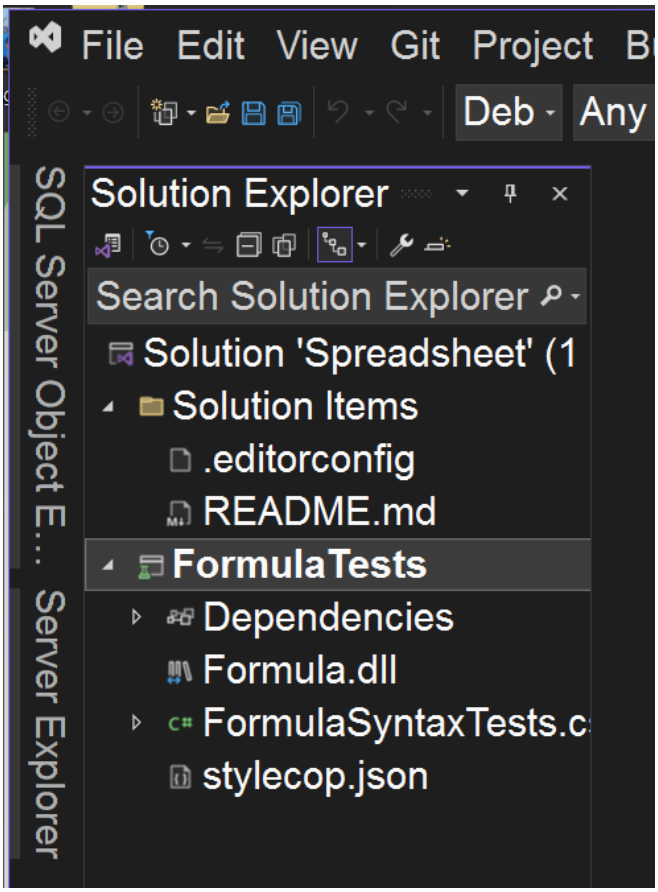
3. In new folder, you should see a file called Spreadsheet.sln - that's the Visual Studio **solution** file that represents a set of related C# projects which are themselves a set of C# programs/files.

4. Double-click the .sln file and it should open in Visual Studio.

   Alternatively, you can run Visual Studio manually, then go to File → Open → Project/Solution and browse to the .sln file.

[1] If you are using a Macintosh laptop, you will either need to use a virtual machine (such as Parallels) or remotely access Visual Studio via a CADE lab remote desktop. https://www.cade.utah.edu/remote-desktop-access/

6.1 The Solution Explorer



You should see a sub-window in Visual Studio called the **Solution Explorer** (usually on the right side, though you can move it to the left if you like). The Solution Explorer shows the files and structure of your C# solution. This solution (Spreadsheet) has one project in it: FormulaTests. Again, think of the solution as a complex program (with multiple parts), and projects within it as sub-components.

6.1.1 Formula.dll

The project contains a dll (pre-compiled code) for a Formula class. There is no C# source code since it has already been compiled. This dll contains the classes that we will be unit testing in this lab and in PS1.

6.1.2 FormulaTests Project

The FormulaTests project is a unit testing project, so it's already set up to write and run unit tests easily.

6.1.3 FormulaTests → Dependencies

The Formula Tests project uses the DLL. This is called a **dependency** (i.e., the project references the Formula code so that it can access the classes and methods contained in the pre-compiled dll.)

6.2 A Simple Unit Test

The Formula Tests project is where we'll be working to write tests for the Formula.

In the Solution Explorer, expand the FormulaTests project, then double-click FormulaSyntaxTests.cs file. This file contains C# code with a unit testing class and a couple of simple test methods already written.

As with all software development, you should take some time to read the comments and code in this file. If something is unclear, bring your questions to your peers, the TAs, and/or the professor.

6.3 Testing For an Invalid Empty Formula

Let's examine the first unit test:

```
[TestMethod]
[ExpectedException(typeof(FormulaFormatException))]
public void FormulaConstructor_TestNoTokens_Invalid()
{
  _ = new Formula( "" );
}
```

Notice a few things about this code:

1. `[TestMethod]` - this Attribute (tag) appears above a method to indicate the code contains a unit test. This attribute is necessary (it tells the testing environment what methods to run when you execute the "Run All" command).

2. `[ExpectedException ...]` - this attribute indicates that this test is expected to throw an exception. If the exception is not thrown by the code within the method body, the test will fail. The test will pass if the right type of exception is thrown.

3. `new Formula( "" );` - this tries to create a new Formula object by passing an empty string to the constructor.

4. The _ (underscore) takes the place of a variable declaration and tells the compiler that we explicitly aren't using the formula object after it is created. This is because we're just testing the constructor, and we don't need the resulting object.

Why is an exception expected?

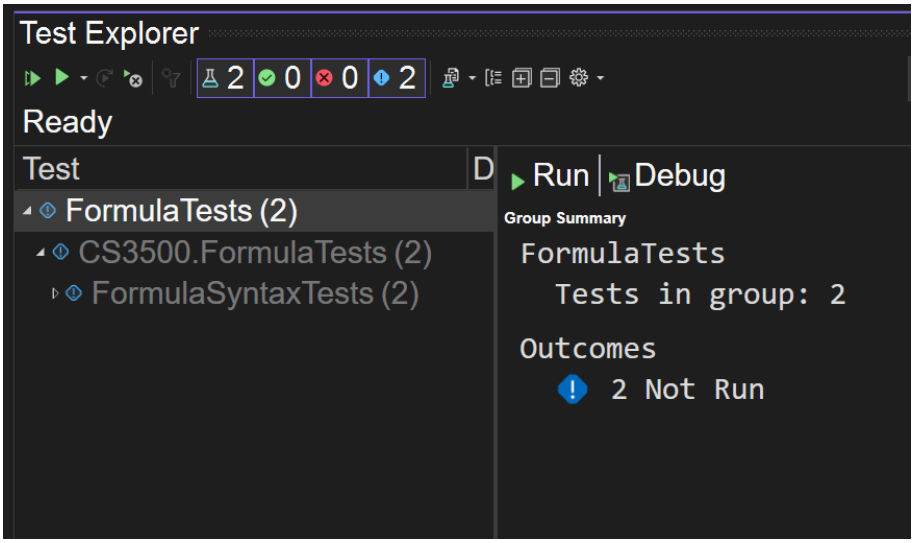
Don't exceptions mean something went wrong? Shouldn't the test fail when exceptions are thrown?

Yes, exceptions do mean that something unexpected occurred, but in a good software specification, we need to know what will happen when something goes wrong. Take a look at the assignment specifications, specifically the One Token Rule. This rule states that a Formula must have at least one token. Our test tries to create a Formula with an empty string, thus no tokens, so it's breaking that rule (on purpose). The Formula specification states that if any of the syntax rules are broken, an exception should be thrown. The test expects an exception to make sure that when the One Token Rule is broken, the Formula constructor throws the right exception.

Answer question 1 on your worksheet.

Now take a look at the other test: "1 + 1" is a valid expression for the Formula constructor and an exception should not be thrown. By leaving off the [ExpectedException] tag we are implicitly saying the code inside the test will not throw an exception.

6.4 Running The Tests



To run the unit tests in Visual Studio, go to the Test menu → Run All Tests. This should open a new sub-window (the Test Explorer), and start to compile and run your tests, which may take a few seconds.

In the Test Explorer window, you should see FormulaTests. Expand it, and the other FormulaTests inside it, and you should see FormulaSyntaxTests (2) with a green check mark. This means the FormulaSyntaxTests class has 2 tests and both of them passed. Expand FormulaSyntaxTests to see the two test methods in that class. Try double-clicking on the test names to take you to the source code for that test - this will be more useful when you have a lot more tests.

6.5 Writing Tests

Let's write a new simple test.

Go to the FormulaConstructor_TestFirstTokenNumber_Valid test and copy/paste it (with the header comment).

> **Important:** Although copy/pasting can be quite useful, always be careful to:
>
> 1. **update the name** of the method,
> 2. the **documentation**,
> 3. and **add/remove any [ExpectedException] attributes**.

In your new test, construct a new Formula object with the following expression string: "(1 + 1)". Note that this is the same expression as the other test (the one you copied), except that it has parentheses around it.

Discuss as a class: does this expression violate any syntax rules?

Using your answer to the previous question, determine whether or not you need the [ExpectedException ...] tag above your test. Then either put the tag there, or leave it off based on what you determine.

Discuss as a class: Naming test methods something meaningful is extremely important so that the reader of your test knows what it does. What is a better name for this new test that follows the MS Test Best Practices? The name should briefly indicate what method is being tested, what the test is doing, and if the test is validating a valid formula or showing that an invalid formula throws the correct exception (see the other provided tests).

- Answer question 2 on your worksheet.
- Run all of your tests again and check the results; they should all pass.
- If you haven't already done so, rename your test.

6.6 Testing The Other Formula Classes

The provided solution comes with a Formula.dll, which contains pre-compiled code. There is not just one, but **three different Formula implementations** in this dll. Two of those implementations were written by past students of this class, and one was written by the course staff. Since we can't have different classes with the same name, the three of them are in different namespaces. A namespace essentially disambiguates class names.

Notice near the top of FormulaSyntaxTests.cs, this line:

```
using CS3500.Formula1;
```

That means we are using the namespace CS3500.Formula1, which contains one of the Formula implementations. Let's use the second implementation instead. Change the using line to this:

```
using CS3500.Formula2;
```

Now all of our statements that refer to the Formula type will actually refer to the Formula2.Formula class (different from the Formula1.Formula class) - **we don't need to change any other code**!

- Run all of the tests again.
- Answer question 3 on your worksheet.

Now change the using statement to use the CS3500.Formula3 namespace and run the tests again.

Our new test should fail on the 3rd Formula implementation.

6.7 Use the Test Explorer:

Discuss as a class: Is our test wrong or is the Formula3 implementation wrong?

Let's see if we can gather a little more information. In the Test Explorer, click on the test that failed to see the summary as you did before. For what reason did the test fail? It can show you whether an assertion failed, if the wrong kind of exception was thrown, etc. If an exception was thrown, it will even show you the exception's message.

- Answer question 4 on your worksheet.

When a test fails, it is **critically** important that you understand the specification you're testing. It's possible we just wrote a bad test, or it's possible the test found a bug (meaning Formula3 is wrong). Triple-check your understanding of the Formula specification.

Discuss as a class: Is the expression "(1 + 1)" a valid expression? Alternatively, is it invalid and an exception should be expected?

7 Bug Report

Part of your task in the first assignment is to write a bug report for all the bugs you find in any of the three Formula implementations through testing. Since we've found our first bug, let's think about what bug report we might make. Take a look at the assignment's bug report instructions to see how you should format and write one.

7.1 How do we know what the bug is or how to describe it?

Look at the other provided test: FormulaConstructor_TestFirstTokenNumber_Valid. Notice that there is a very small difference (the parentheses) between that test and our new failing test. What does that tell us about the likely nature of the bug? It's most likely a problem involving parentheses, so a reasonable first assumption for our bug description could be something like:

> **Hypothesis**: "The Formula constructor fails to construct a Formula when the first token of a valid formula is an opening parentheses".

Keep in mind it may not actually be directly related to parentheses. Maybe the bug is that Formula3.Formula class can't handle anything other than a number in the last token... You may never know the real answer until you have access to the actual code, though you can still do some more extensive testing to get a more comprehensive view of what is likely.

Although we have already thought about what the bug report might say, you will want to **wait until you've written a full comprehensive set of tests** before completing that part of the assignment.

8 Write More Tests

With the remaining time in the lab, start working on the assignment. Review the Formula specification and think up some more tests you might want to write. You can discuss general ideas with classmates, but **do not share the tests themselves** - those must be written **individually** by you.